



AWS Academy Cloud Developing
Module 11 Student Guide
Version 2.0.6

200-ACCDEV-20-EN-SG

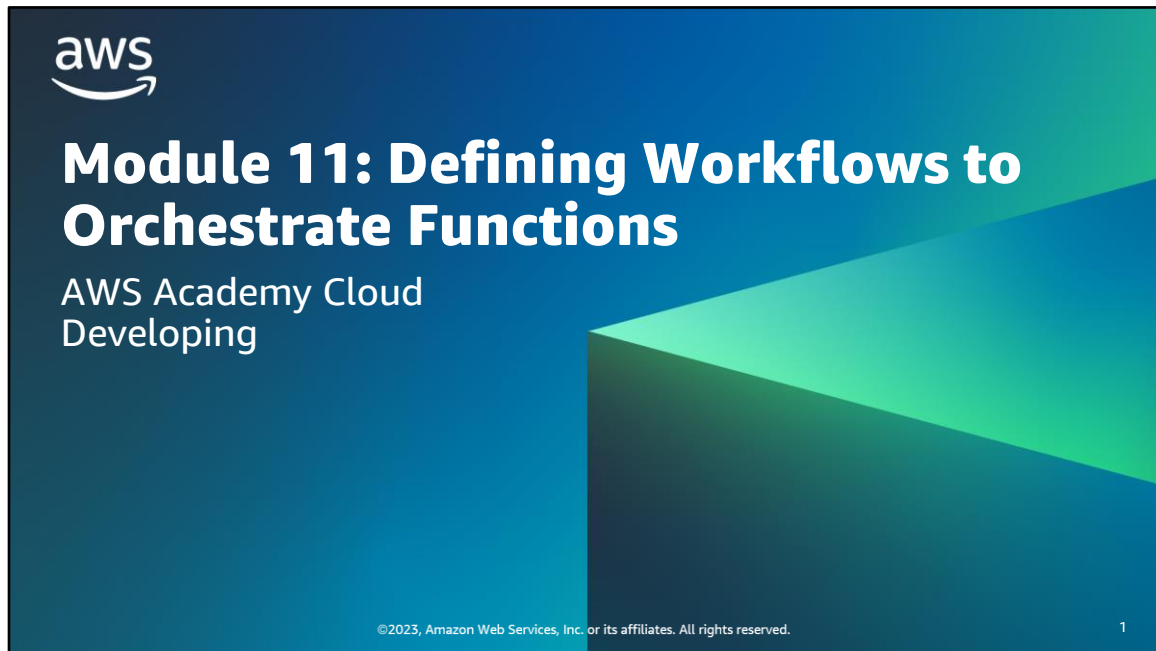
© 2024, Amazon Web Services, Inc. or its affiliates. All rights reserved.

This work may not be reproduced or redistributed, in whole or in part,
without prior written permission from Amazon Web Services, Inc.
Commercial copying, lending, or selling is prohibited.

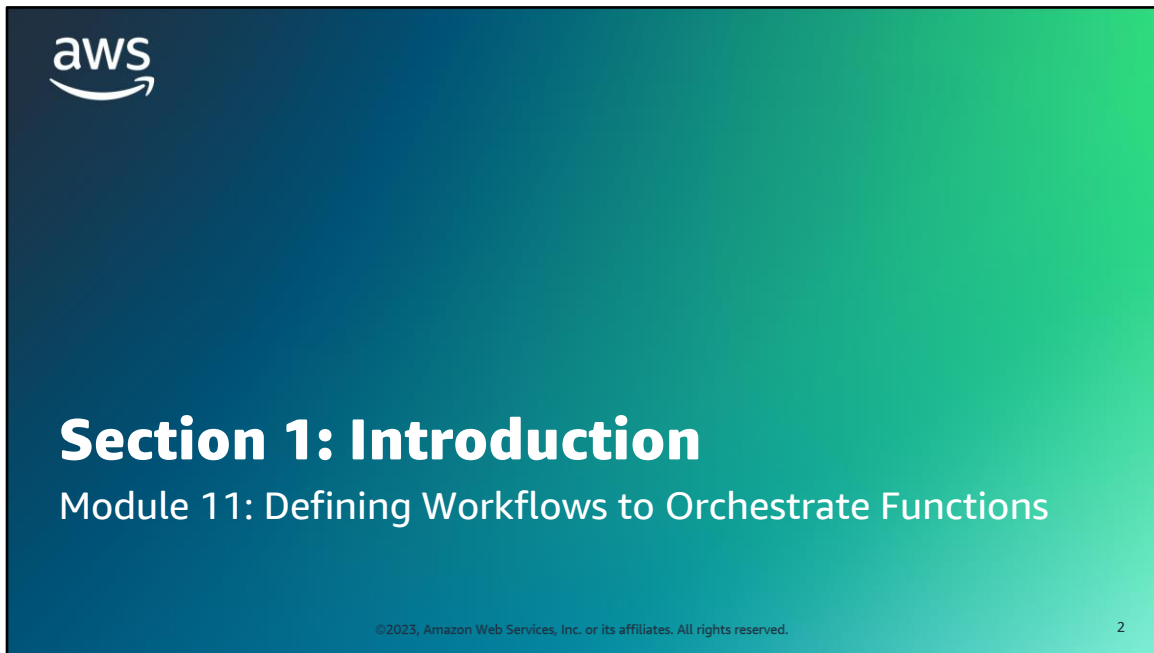
All trademarks are the property of their owners.

Contents

Module 11: Defining Workflows to Orchestrate Functions	4
--	---



Welcome to Module 11: Defining Workflows to Orchestrate Functions.



Section 1: Introduction.

Module objectives

At the end of this module, you should be able to do the following:

- Recognize the dynamics of workflow coordination in distributed applications
- Describe AWS Step Functions
- Identify state types
- Indicate common use cases for Step Functions
- Recall Step Functions API operations



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

3

At the end of this module, you should be able to do the following:

- Recognize the dynamics of workflow coordination in distributed applications
- Describe AWS Step Functions
- Identify state types
- Indicate common use cases for Step Functions
- Recall Step Functions API operations

Module overview

Sections

1. Introduction
2. Coordinating tasks in distributed applications
3. Introducing Step Functions
4. State types
5. Step Functions use cases
6. Step Functions API

Demonstration

- Creating Simple Calculators by Using Step Functions

Lab

- Lab: Orchestrating Serverless Functions with Step Function



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

4

This module includes the following sections:

1. Introduction
2. Coordinating tasks in distributed applications
3. Introducing Step Functions
4. State types
5. Step Functions use cases
6. Step Functions API

This module also includes:

- A demonstration about creating simple calculators by using Step Functions
- A lab where you will orchestrate serverless functions with Step Functions

Finally, you will complete a knowledge check to test your understanding of key concepts covered in this module.

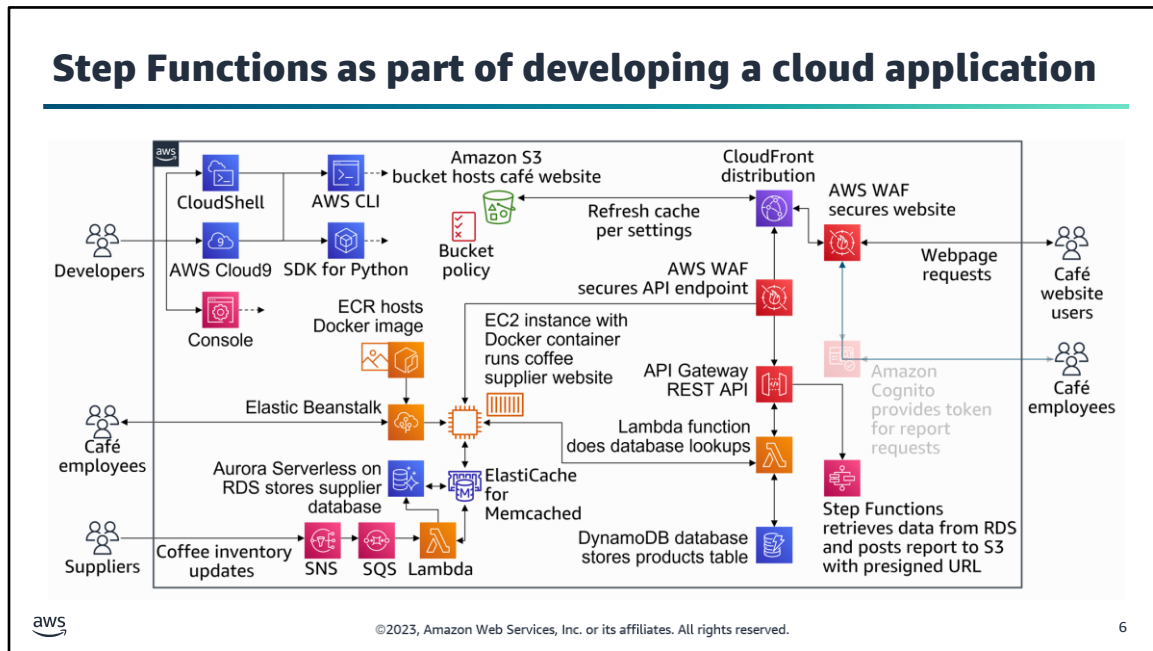
Café business requirement

Now that the coffee inventory is automatically updated on the café website, Frank would like to be able to request reports with the latest inventory information. Sofía thinks she can use Step Functions to coordinate the steps to generate a report.



Now that the coffee inventory is automatically updated on the café website, Frank would like to be able to request reports with the latest inventory information. Sofía thinks she can use Step Functions to coordinate the steps to generate a report.

The inventory information is stored in an Amazon Relational Database Service (Amazon RDS) database, and Sofía can create an AWS Lambda function to format the data into a report. She would like to be able to use the Amazon Simple Notification Service (Amazon SNS) to send Frank an email when the report is ready. Step Functions can orchestrate those tasks into a workflow.



The diagram on this slide gives an overview of the application that you will build through the labs in this course. The highlighted portions are relevant to this module.

As highlighted in the diagram, you will create a Step Functions workflow and integrate it with a REST API. The workflow will coordinate the tasks of getting data from the database, generating a report, posting it to Amazon Simple Storage Service (Amazon S3), and sending a notification through Amazon SNS.

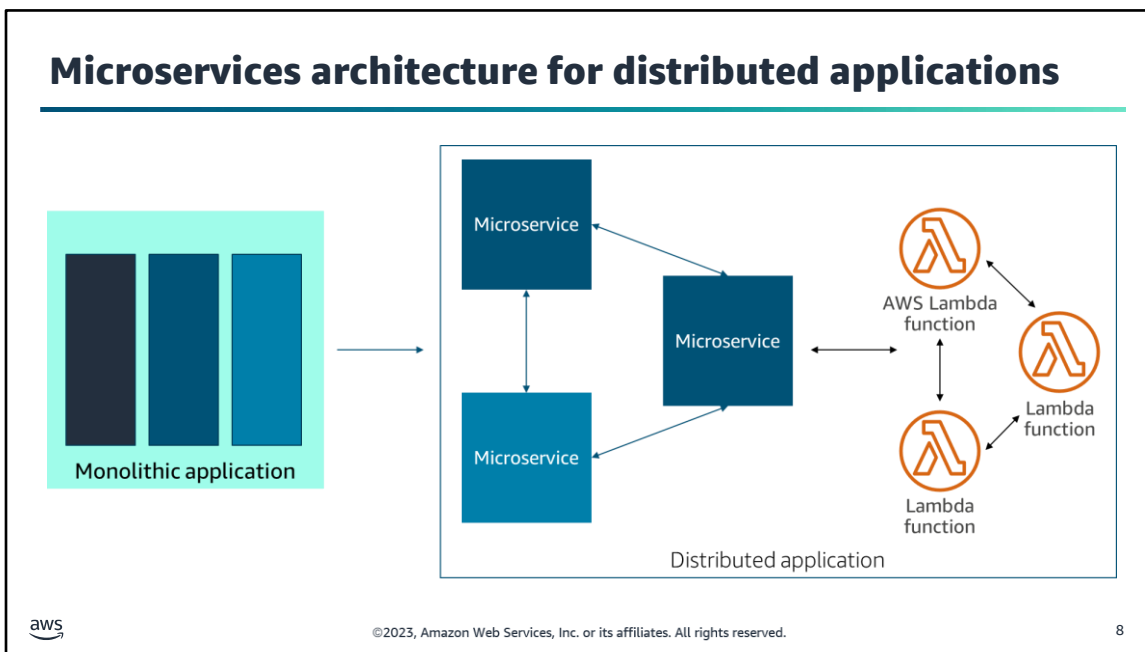


Section 2: Coordinating tasks in distributed applications

Module 11: Defining Workflows to Orchestrate Functions

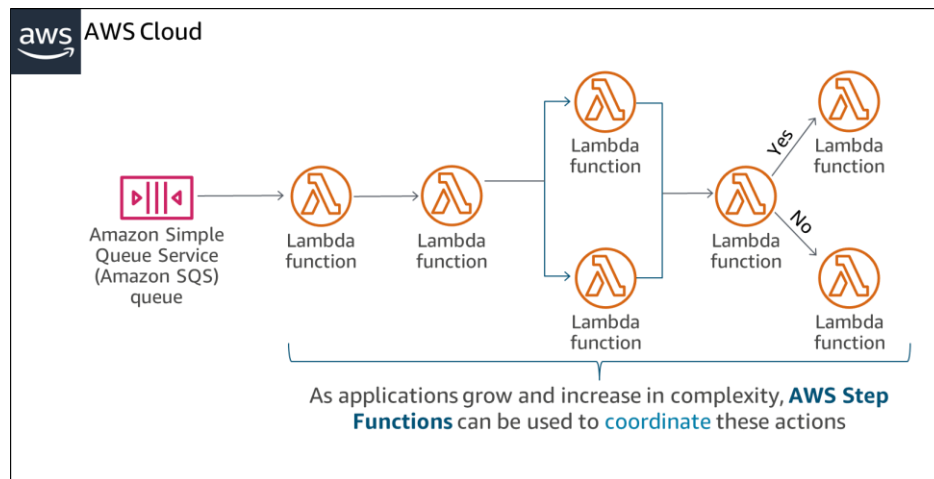
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved. 7

Section 2: Coordinating tasks in distributed applications.



In modern cloud architecture, applications are decoupled into smaller, independent components that are called microservices, which are easier to develop, deploy, and maintain. With microservices, you can build distributed applications from individual components that each perform a discrete function or *task*. You can use a microservices architecture to deploy, scale, and update components of your application quickly.

Challenges with distributed applications



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

9

As your application grows in size, the number of components increases and you can have many patterns to run tasks. For example, consider a serverless application that is built by using AWS Lambda functions. You might want to invoke a Lambda function immediately after another function, and only if the first function runs successfully. You might want two functions to be invoked in parallel and then feed the combined results into a third function. Or you might want to choose which of two functions is invoked based on the output of another function.

Function invocation can result in an error for several reasons. Your code might raise an exception, time out, or run out of memory. The runtime that runs your code might encounter an error and stop. When an error occurs, your code might have run completely, partially, or not at all. In most cases, the client or service that invokes your function automatically retries if it encounters an error. Therefore, your code must be able to process the same event repeatedly without unwanted effects. If your function manages resources or writes to a database, you must handle cases where the same request is made several times.

You need a way to coordinate the components of your application. This coordination layer is necessary to provide the capability to scale automatically in response to changing workloads, and to handle errors and timeouts. It must also maintain the state of your application while it's running. For example, it must track what step it's in and store data that is moving between the steps of your workflow. These features will help you build and operate your applications. You also want visibility into your application so that you can troubleshoot errors and track performance.

Section 2 key takeaways



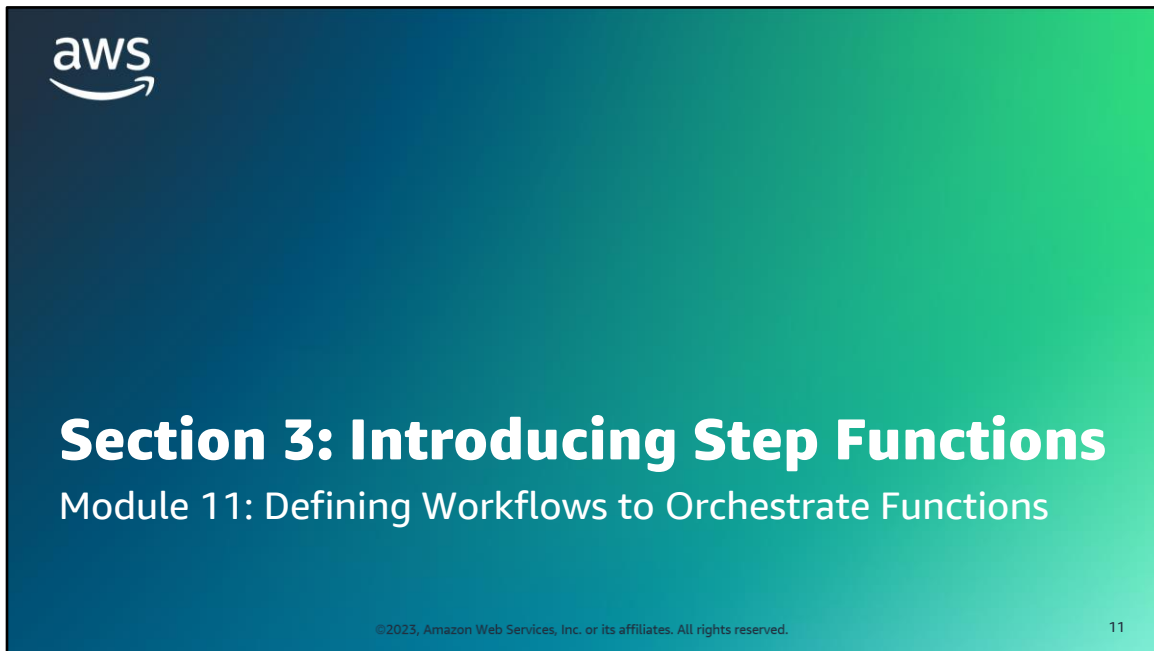
- **Microservices** are smaller, independent components that make up an application in modern cloud architecture.
- You can use microservices to quickly **deploy, scale, and update** application components.
- A **coordination layer** is necessary to handle **automatic scaling, errors, and timeouts**.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

10

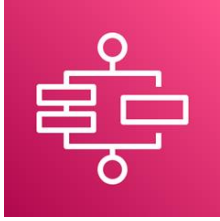
The following are the key takeaways from this section of the module:

- Microservices are smaller, independent components that make up an application in modern cloud architecture.
- You can use microservices to quickly deploy, scale, and update application components.
- A coordination layer is necessary to handle automatic scaling, errors, and timeouts.



Section 3: Introducing Step Functions.

AWS Step Functions



AWS Step Functions

Serverless orchestration service that you can use to combine Lambda functions and other AWS services to build critical applications

- Coordinates components of distributed applications and microservices by using visual workflows
- Automatically triggers and tracks each step, and retries when errors occur
- Logs the state of each step



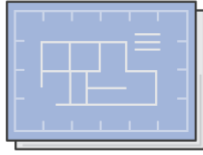
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

12

AWS Step Functions is a web service that you can use to coordinate components of distributed applications and microservices by using visual workflows. Step Functions provides a reliable way to coordinate components and step through the functions of your application. Step Functions offers a graphical console so that you can visualize the components of your application as a series of steps. It automatically triggers and tracks each step, and it also retries when errors occur, so your application runs in order and as expected. Step Functions logs the state of each step, so you can diagnose and debug problems quickly.

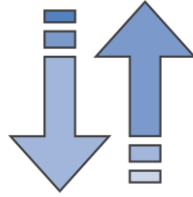
For more information about Step Functions, see <https://aws.amazon.com/step-functions/>.

Benefits of Step Functions



Productivity

Build and update applications quickly



Agility

Scale and recover reliably



Resiliency

Evolve applications easily



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

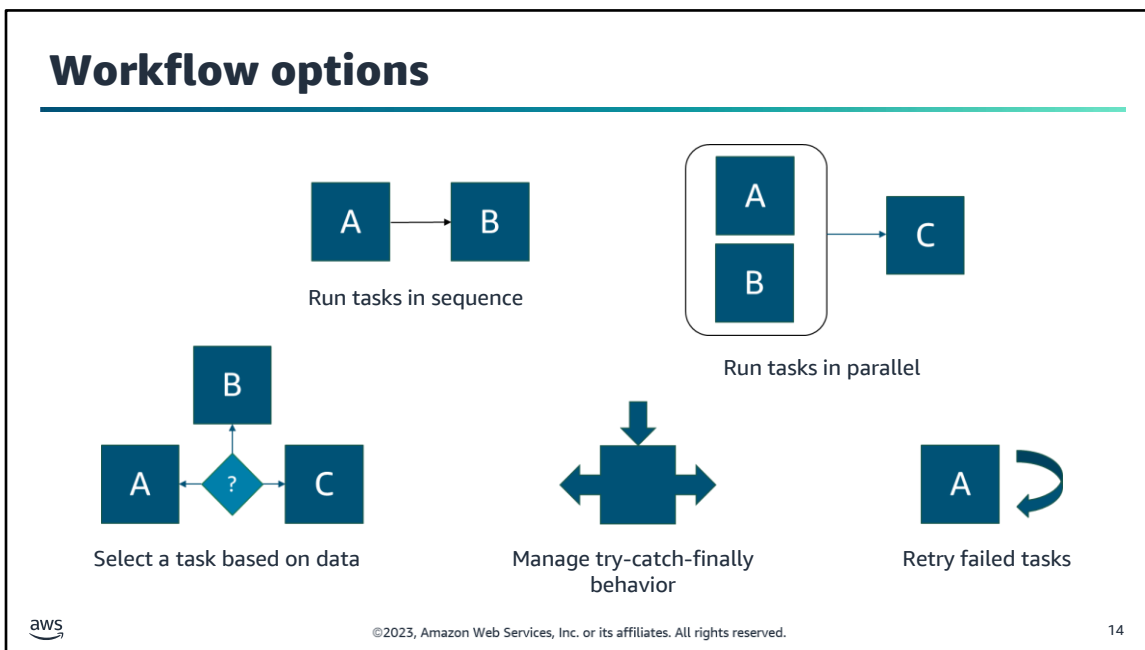
13

You can use Step Functions to do the following:

- **Build and update applications quickly:** You can use Step Functions to build visual workflows that provide fast translation of business requirements into technical requirements. You can build applications in minutes. When needs change, you can exchange or reorganize components without customizing any code.
- **Scale and recover reliably:** Step Functions automatically scales the operations and underlying compute to run the steps of your application for you in response to changing workloads. Step Functions scales automatically to help ensure that the performance of your application workflow remains consistent as the frequency of requests increases.
- **Evolve applications easily:** Step Functions manages state, checkpoints, and restarts for you to make sure that your application runs in order and as expected. It has built-in try/catch, retry, and rollback capabilities to deal with errors and exceptions automatically.

For more information about Step Functions features, see

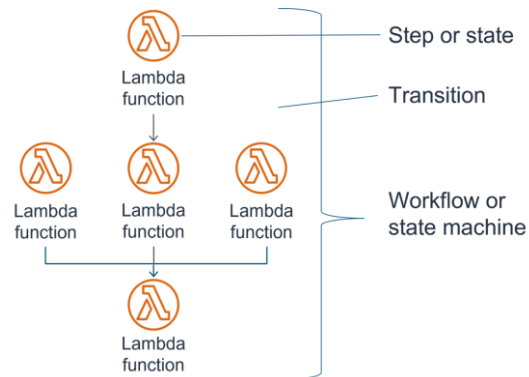
<https://aws.amazon.com/step-functions/features/>.



Step Functions manages the logic of your application for you and implements basic data types, such as running tasks sequentially and in parallel, branching, and timeouts. This process removes extra code that might be repeated in your microservices and functions. Step Functions automatically handles errors and exceptions with built-in try/catch and retry, whether the task takes seconds or months to complete. You can automatically retry failed or timed-out tasks, respond differently to different error types, and recover gracefully by returning to designated cleanup and recovery code.

How Step Functions works

- You define your **workflow** (or state machine) as a series of **steps** (or states) and **transitions**.
- **Tasks** do all of the work in a state machine.
- Each state machine starts with an **input**. The state transforms the input and passes the **output** to the next state.
- You can **visualize** and **track** workflows in the Step Functions console.



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

15

With Step Functions, you define a *workflow* as a series of *steps* and *transitions* between each step. A workflow is also called a *state machine*, and states are elements in a state machine. A state machine is an object that has a set number of operating conditions that depend on the object's previous condition to determine output. As a result, individual states can make decisions based on their input, perform actions, and pass output to other states.

Tasks do all of the work in your state machine. As you will learn in the next section, a task can perform work by using an activity or an AWS Lambda function. Alternatively, it can pass parameters to the API actions of other services. Each state machine starts with an input. The state transforms the input and passes the output to the next state.

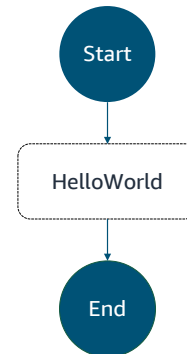
You can use the Step Functions console to visualize your state machine, and it provides near-real-time information on your state machine tasks.

Defining and visualizing workflows

Define the workflow in JSON by using the Amazon States Language

```
{
  "Comment": "A Hello world Example",
  "StartAt": "HelloWorld",
  "States": {
    "HelloWorld": {
      "Type": "Pass",
      "Result": "Hello world!",
      "End": true
    }
  }
}
```

Visualize the workflow in the Step Functions console



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

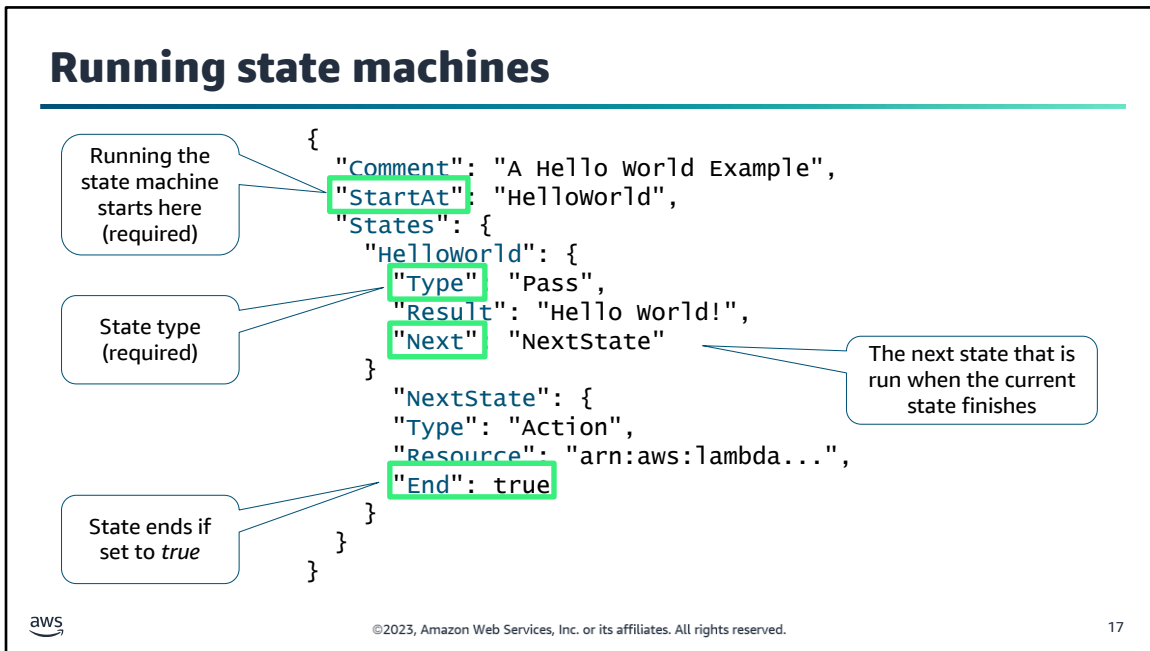
16

You define state machines by using the Amazon States Language (ASL). The Amazon States Language is a JSON-based, structured language. Step Functions then represents the JSON structure in a real-time graphical view so you can visualize your state machine directly in the Step Functions console.

AWS Step Functions Workflow Studio is another way to define state machines. You can use this low-code visual tool to build Step Functions through a guided, interactive interface, so that you can prototype and build workflows faster.

For more information about the Amazon States Language, see <https://docs.aws.amazon.com/step-functions/latest/dg/concepts-amazon-states-language.html>.

For more information about AWS Step Functions Workflow Studio, see <https://docs.aws.amazon.com/step-functions/latest/dg/workflow-studio.html>.



When a state machine is launched, the system begins with the state that is referenced in the top-level **StartAt** field. After Step Functions runs a state, it uses the value of the **Next** field to determine the next state to advance to. This step is called a *transition*.

A terminal state is a *Succeed* state, a *Fail* state, or if the **End** field is set to *true*. When the system reaches a terminal state or if a runtime error occurs, the state stops running and returns a result.

For more information, see the following sections of the *AWS Step Functions Developer Guide*:

- Transitions: <https://docs.aws.amazon.com/step-functions/latest/dg/concepts-transitions.html>.
- Common State Fields: <https://docs.aws.amazon.com/step-functions/latest/dg/amazon-states-language-common-fields.html>.

Section 3 key takeaways



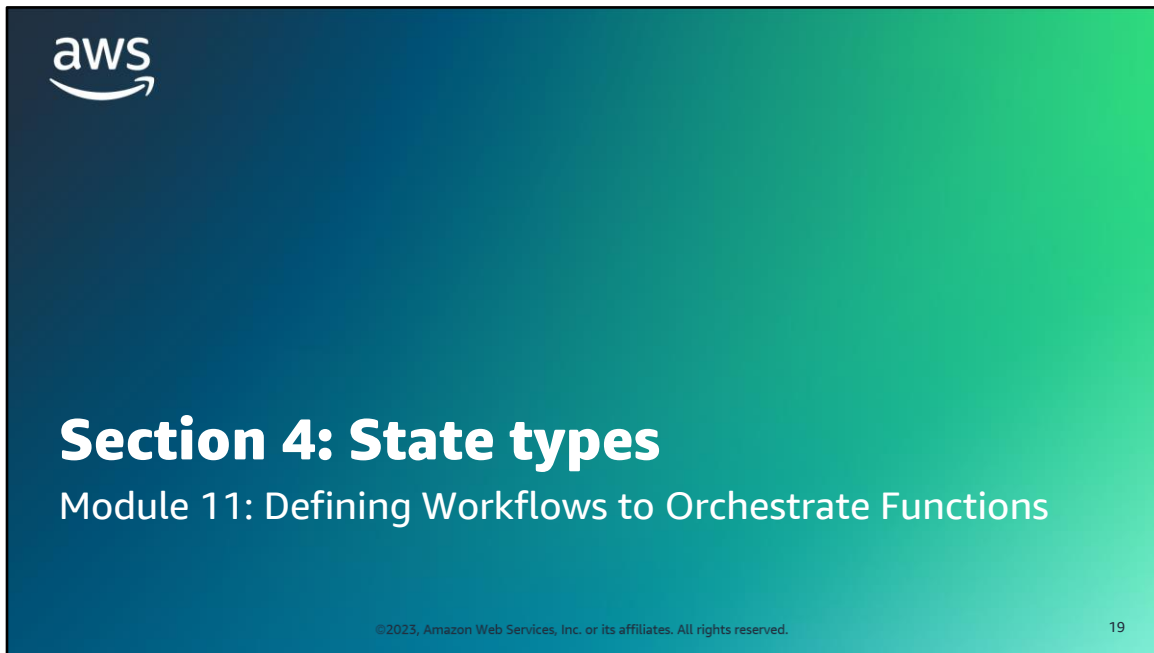
- **Step Functions** is a service that you can use to coordinate components of distributed applications and **microservices** by using visual workflows.
- A **state machine** is an object that has a set number of operating conditions that depend on the object's **previous condition** to determine the **output**.
- State machines are **defined** by using the **Amazon States Language**.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

18

The following are the key takeaways from this section of the module:

- Step Functions is a service that you can use to coordinate components of distributed applications and microservices by using visual workflows.
- A state machine is an object that has a set number of operating conditions that depend on the object's previous condition to determine output.
- State machines are defined by using the Amazon States Language.



Section 4: State types.

State types

Task	A single unit of work
Choice	Adds branching logic to a state machine
Fail	Stops running a state and marks it as a failure
Succeed	Stops running a state successfully
Pass	Passes its input to its output, without performing work
Wait	Delays from continuing for a specified time
Parallel	Creates parallel branches for running states
Map	Dynamically iterates steps



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

20

A finite state machine can express an algorithm as a number of states, their relationships, and their input and output. States are elements in your state machine. Individual states can make decisions based on their input, perform actions, and pass output to other states.

States can perform a variety of functions in your state machine:

- Do some work in your state machine (a *Task* state)
- Make a choice of which branch to run (a *Choice* state)
- Stop running a state with a failure or success (a *Fail* state or *Succeed* state)
- Pass its input to its output or inject some fixed data (a *Pass* state)
- Provide a delay for a certain amount of time or until a specified time and date (a *Wait* state)
- Begin running parallel branches (a *Parallel* state)
- Dynamically iterate steps (a *Map* state)

In this section, you will learn about these different state types.

For more information about state types, see <https://docs.aws.amazon.com/step-functions/latest/dg/concepts-states.html>.

Task state



Amazon Elastic
Compute Cloud
(Amazon EC2)

```
"ActivityState": {
  "Type": "Task",
  "Resource": "arn:aws:states:us-east-1:123456789012:activity:HelloWorld",
  "TimeoutSeconds": 300,
  "HeartbeatSeconds": 60,
  "Next": "NextState"
}
```



Amazon Elastic
Container Service
(Amazon ECS)

```
"ActivityState": {
  "Type": "Task",
  "Resource": "arn:aws:states:us-east-1:123456789012:activity:HelloWorld",
  "TimeoutSeconds": 300,
  "HeartbeatSeconds": 60,
  "Next": "NextState"
}
```



Lambda
function

```
"LambdaState": {
  "Type": "Task",
  "Resource": "arn:aws:lambda:us-east-1:123456789012:function:HelloWorld",
  "Next": "NextState"
}
```



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

21

Tasks do all of the work in your state machine. A task performs work by using an activity or a Lambda function, or by passing parameters to the API actions of other services. The Amazon States Language represents tasks by setting a state's type to *Task* (that is, "Type": "Task"). It also provides the task with the Amazon Resource Name (ARN) of the activity or Lambda function.

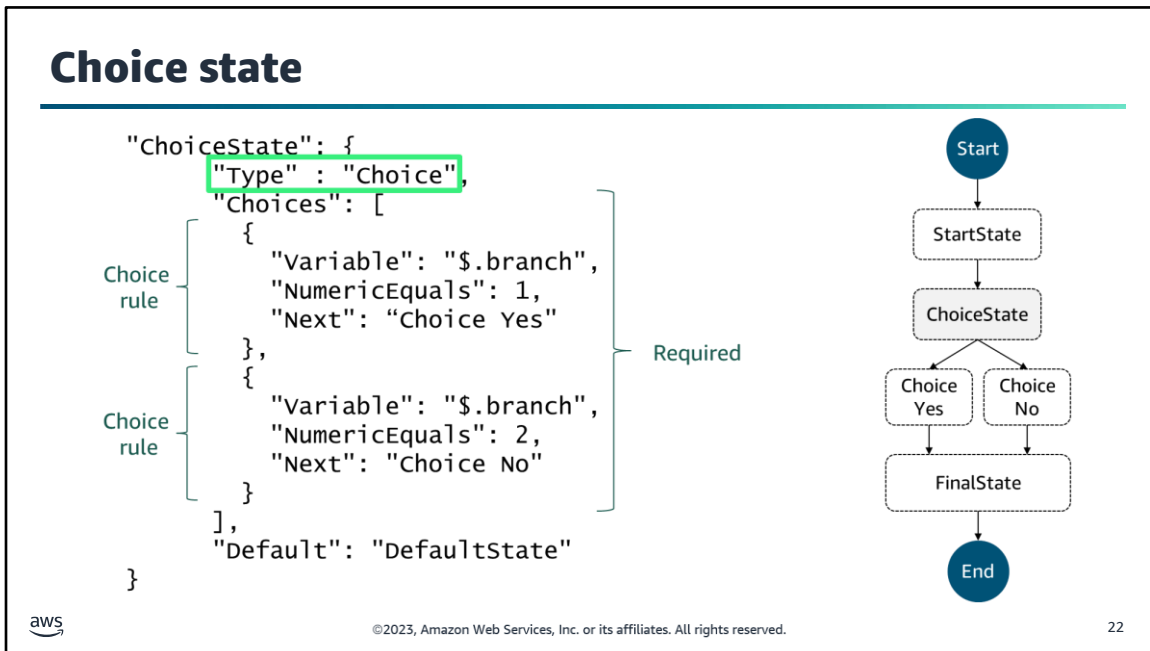
An activity consists of program code that waits for an operator to perform an action or to provide input. You can host activities on Amazon Elastic Compute Cloud (Amazon EC2), Amazon Elastic Container Service (Amazon ECS), or even mobile devices. Activities poll Step Functions by using the `GetActivityTask`, `SendTaskSuccess`, `SendTaskFailure`, and `SendTaskHeartbeat` API actions.

Step Functions can invoke Lambda functions directly from a task state. Step Functions also integrates with some AWS services, including the ability to directly call and pass parameters to the API of those services. You can use Step Functions to coordinate these services directly from the Amazon States Language.

The two examples show the *Task* state. The first example performs an activity that is named *HelloWorld* (which is identified by the activity ARN in the **Resource** field). The second example performs a Lambda function (which is identified by the Lambda function ARN).

For more information about the Task state, see <https://docs.aws.amazon.com/step-functions/latest/dg/amazon-states-language-task-state.html>.

For more information about how Step Functions integrates with other AWS services, see <https://docs.aws.amazon.com/step-functions/latest/dg/concepts-connectors.html>.



A Choice state ("Type": "Choice") adds branching logic to a state machine. A Choice state must have a **Choices** field whose value is a non-empty array, and whose element is an object called a Choice Rule.

A Choice Rule contains the following:

- A *comparison*: Two fields that specify an input variable to be compared, the type of comparison, and the value to compare the variable to.
- A *Next* field: The value of this field must match a state name in the state machine.

In the example, the first choice rule checks whether the numerical value is equal to 1. The second choice rule checks whether the numerical value is equal to 2.

Note: The "\$" symbol in these examples is an indicator of a variable.

For more information about the Choice state, see

<https://docs.aws.amazon.com/step-functions/latest/dg/amazon-states-language-choice-state.html>.

Fail and Succeed states

```
"FailState": {  
  "Type": "Fail",  
  "Cause": "Invalid response.",  
  "Error": "ErrorA"  
}
```

Fail state

```
"SuccessState": {  
  "Type": "Succeed"  
}
```

Succeed state



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

23

A Fail state ("Type": "Fail") stops running the state machine and marks it as a failure. The Fail state requires the **Type** field (which specifies the state's type). In addition, the Fail state offers the following optional fields:

- **Cause:** Provides a custom failure string that can be used for operational or diagnostic purposes.
- **Error:** Provides an error name that can be used for error handling (retry/catch), operational, or diagnostic purposes.

Because Fail states always exit the state machine, they have no **Next** field and they do not require an **End** field. The slide provides examples of a Succeed state and a Fail state.

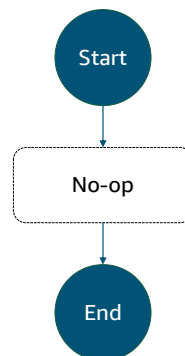
A Succeed state ("Type": "Succeed") stops running a state successfully. The Succeed state is a useful target for Choice state branches that don't do anything except stop running the state. Because Succeed states are terminal states, they have no **Next** field, and they don't need an **End** field.

For more information, see the following sections of the *AWS Step Functions Developer Guide*:

- Succeed: <https://docs.aws.amazon.com/step-functions/latest/dg/amazon-states-language-succeed-state.html>.
- Fail: <https://docs.aws.amazon.com/step-functions/latest/dg/amazon-states-language-fail-state.html>.

Pass state

```
"No-op": {  
  "Type": "Pass",  
  "Result": {  
    "x-datum": 0.381018,  
    "y-datum": 622.2269926397355  
  },  
  "ResultPath": "$.coords",  
  "Next": "End"  
}
```



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

24

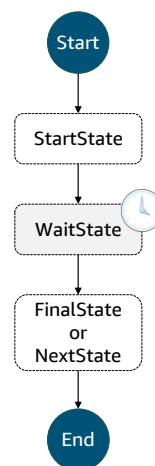
A Pass state ("Type": "Pass") passes its input to its output, without performing work. Pass states are useful when constructing and debugging state machines.

This example shows a state machine that consists of a single Pass state. When this state machine runs, it injects some fixed data, which might be useful for testing purposes.

For more information about the Pass state, see <https://docs.aws.amazon.com/step-functions/latest/dg/amazon-states-language-pass-state.html>.

Wait state

```
"wait_ten_seconds": {  
  "Type": "Wait",  
  "Seconds": 10,  
  "Next": "NextState"  
}
```



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

25

A Wait state ("Type": "Wait") delays the state machine from continuing for a specified time. You can choose either a relative time (specified in seconds from when the state begins), or an absolute end time (specified as a timestamp).

In this example, the Wait state introduces a 10-second delay into the state machine.

For more information about the Wait state, see <https://docs.aws.amazon.com/step-functions/latest/dg/amazon-states-language-wait-state.html>.

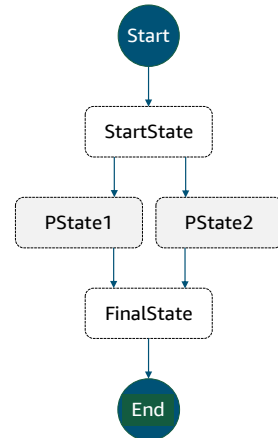
Parallel state

```

"LookupCustomerInfo": {
  "Type": "Parallel"
  "Branches": [
    {
      "StartAt": "PState1",
      "States": {
        "PState1": {...}
      }
    },
    {
      "StartAt": "PState2",
      "States": {
        "PState2": {...}
      }
    }
  ],
  "Next": "NextState"
}

```

Required



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

26

The Parallel state ("Type": "Parallel") creates parallel branches running within your state machine.

A Parallel state causes Step Functions to run each branch—starting with the state named in that branch's **StartAt** field—as concurrently as possible. It will wait until all branches reach a terminal state before it processes the Parallel state's **Next** field.

A branch can fail because of an unhandled error, or by transitioning to a Fail state. If a branch fails for either reason, the entire Parallel state is considered to have failed and all of its branches are stopped.

For more information about the Parallel state, see

<https://docs.aws.amazon.com/step-functions/latest/dg/amazon-states-language-parallel-state.html>.

Map state

```
"validate-All": {
  "Type": "Map",
  "InputPath": "$.detail", } Iterator (required)
  "ItemsPath": "$.shipped",
  "MaxConcurrency": 0,
  "Iterator": {
    "StartAt": "Validate",
    "States": {
      "validate": {
        "Type": "Task",
        "Resource": "arn:aws:lambda:us-east-1:123456789012:function:ship-val",
        "End": true
      }
    }
  },
  "ResultPath": "$.detail.shipped",
  "End": true
}
```



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

27

The Map state ("Type": "Map") creates a set of steps that can be run for multiple entries of an array.

By using a Map state, Step Functions can run the same steps for multiple entries of an array in the state output. The Iterator field is required; this field's value is a reference path that identifies where in the effective input the array field is found. In this slide's example, the iterator is the "InputPath" field.

For more information about the Map state, see <https://docs.aws.amazon.com/step-functions/latest/dg/amazon-states-language-map-state.html>.

Section 4 key takeaways



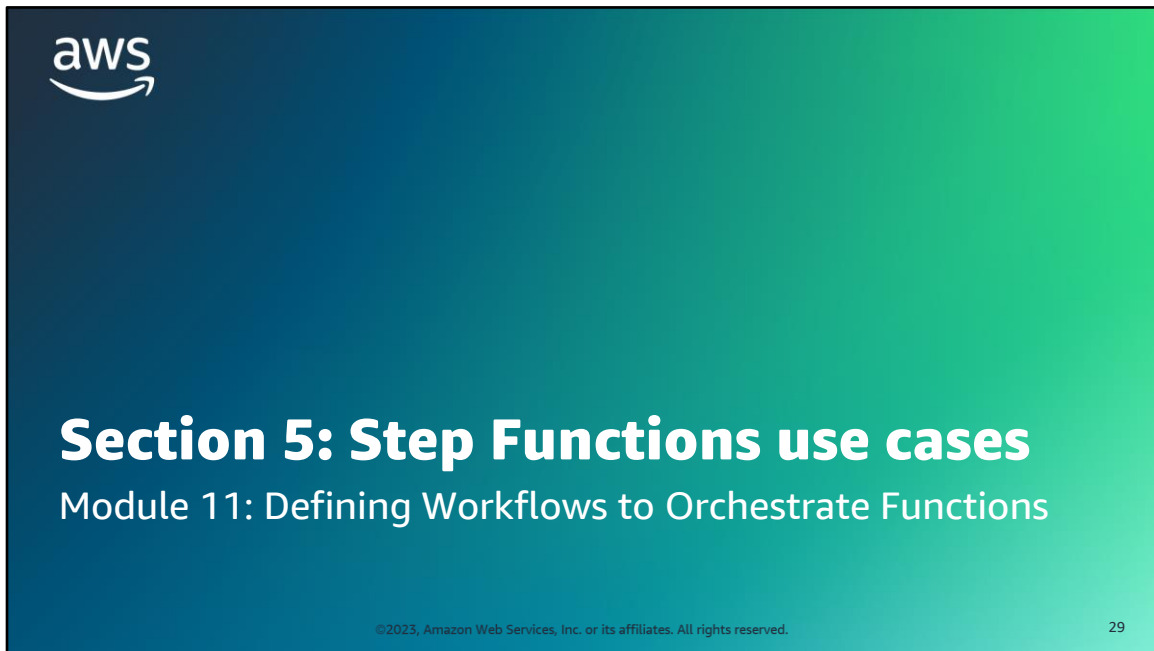
- Individual states can make **decisions** based on their **input**, perform **actions**, and pass and manipulate **output** to other states.
- Add **branching logic** to a state machine by adding a **Choice** state.
- Run multiple functions **concurrently** by using the **Parallel** state.
- **Succeed** and **Fail** states do not require an **End** field.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

28

The following are the key takeaways from this section of the module:

- Individual states can make decisions based on their input, perform actions, and pass and manipulate output to other states.
- Add branching logic to a state machine by adding a Choice state.
- Run multiple functions concurrently by using the Parallel state.
- Succeed and Fail states do not require an End field.



Section 5: Step Functions use cases.

Standard and Express workflows

	Standard	Express
Invocation Rate	2,000 per second	100,000 per second
Transition Rate	4,000 per second	Nearly unlimited
Pricing	Per state transition	Per number and duration of requests for your workflow
Step Functions Activities	Supports Step Functions activities	Does not support Step Functions activities
Integrations and Patterns Support	Supports all service and integration patterns	Supports all service integrations and most patterns



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

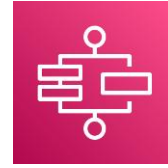
30

Step Functions has two types of workflow: Standard and Express. You define your state machine by using the Amazon States Language regardless of which workflow type you use. However, the state machine behavior when run will vary based on your choice of Standard or Express workflows.

For more information, see Standard vs. Express Workflows at <https://docs.aws.amazon.com/step-functions/latest/dg/concepts-standard-vs-express.html>.

Standard workflows

- Ideal for long-running, auditable workflows
- Can run for up to **1 year**
- Employ an exactly-once model
- Never run more than **once** unless the Retry behavior is specified in ASL
- Are billed according to the number of processed state transitions



AWS Step
Functions



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

31

Standard workflows are ideal for long-running, auditable workflows, because they show a history of previous runs, and provide visual debugging. The full run history can be retrieved by using the Step Functions API for up to 90 days after your run has completed. Standard workflows have an *exactly-once* workflow model, which means that they never run more than once unless you have specified the Retry behavior in ASL. Therefore, they are suitable for non-idempotent actions because their invocations always result in a change of state.

Express workflows

- Ideal for high-volume, event-processing workloads
- Can run for up to **5 minutes**
- Employ an at-least-once (asynchronous) or at-most-once (synchronous) model
- An invocation can run **more than once**
- Are billed according to number and duration of requests, and memory that is consumed



AWS Express
Workflows



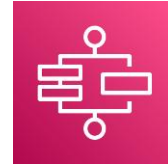
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

32

Express workflows are ideal for high-event-rate workloads, such as streaming data processing and Internet of Things (IoT) data ingestion. Asynchronous Express workflows have an *at-least-once* workflow model, and Synchronous Express Workflows use an *at-most-once* workflow model. Express workflows can run for up to 5 minutes. They are billed based on the number of and duration of requests and memory that is consumed.

Step Functions use cases

- Process data
- Automate tasks
- Modernize a monolithic application
- Orchestrate an application
- Transcode media files
- Sequence batch processing jobs
- Send messages from automated workflows
- Publish events from serverless workflows
- Coordinate container tasks in microservices and serverless applications
- Access databases from serverless workflows
- Sequence steps of machine learning (ML) workflows
- Coordinate extract, transform, and load (ETL) jobs



AWS Step
Functions



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

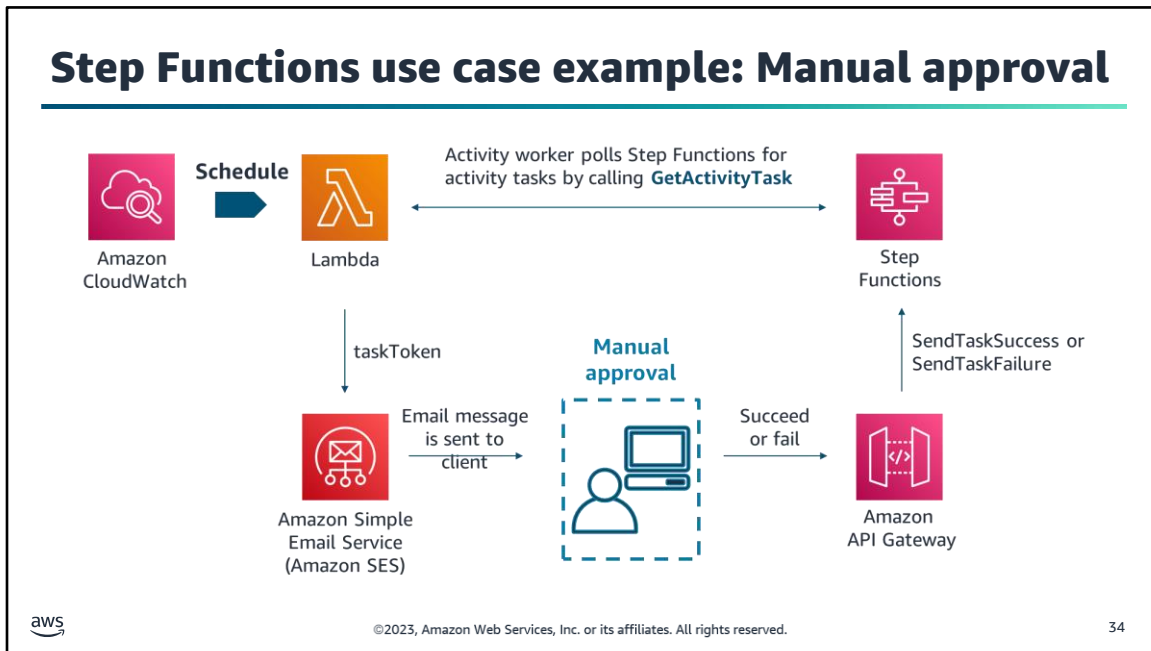
33

Step Functions can be used for a variety of use cases:

- Process data
- Automate tasks
- Modernize a monolithic application
- Orchestrate an application
- Transcode media files
- Sequence batch processing jobs
- Send messages from automated workflows
- Publish events from serverless workflows
- Coordinate container tasks in microservices and serverless applications
- Access databases from serverless workflows
- Sequence steps of machine learning (ML) workflows
- Coordinate extract, transform, and load (ETL) jobs

These use cases can rely on Standard or Express workflows depending on their requirements.

For information about specific use cases, see <https://aws.amazon.com/step-functions/use-cases/>.



In Step Functions, activities are a way to associate code that runs somewhere (an *activity worker*) with a specific task in a state machine. An activity worker can be an application that runs on an EC2 instance, Lambda function, or mobile device. It can be any application that can make an HTTP connection, which is hosted anywhere.

When Step Functions reaches an activity task state, the workflow waits for an activity worker to poll for a task. An activity worker polls Step Functions by using the `GetActivityTask` API action, and it sends the ARN for the related activity.

`GetActivityTask` returns a response, including input (a string of JSON input for the task) and a `taskToken` (a unique identifier for the task). A task that is waiting for a `taskToken` will wait until the invocation reaches the one-year service quota. It relies on a Heartbeat threshold, which is specified in the state machine's ASL definition.

A `taskToken` is a token that represents a specific task and is generated by Step Functions when tasks are assigned to a worker. After the activity worker completes its work, it can provide a report of its success or failure by using `SendTaskSuccess` or `SendTaskFailure`. These two calls use the `taskToken` that `GetActivityTask` provides to associate the result with that task.

One common use case for activities is to manage a workflow that has a manual approval step, such as a promotion approval. In this example, Amazon CloudWatch invokes Lambda on a schedule. Lambda (the activity worker) polls Step Functions for an activity task by calling `GetActivityTask`. First, Lambda successfully calls the API action. Next, the activity is vended to Lambda as a string of JSON input for the task and a `taskToken` for callback. Then, Lambda calls the Amazon Simple Email Service (Amazon SES) messaging service. The messaging service sends an email notification to a manager to either approve or deny an employee's promotion. The response is recorded as either a success (approve) or failure (deny) and is sent as an API request to Amazon API Gateway. API Gateway calls either `SendTaskSuccess` or `SendTaskFailure` and uses the vended token.

This example shows a serverless application. The code that fetches the work and acquires the token is separate from the code that responds with the completion status and returns the token.

For more information, see the Implementing Serverless Manual Approval Steps in AWS Step Functions and Amazon API Gateway blog post at <https://aws.amazon.com/blogs/compute/implementing-serverless-manual-approval-steps-in-aws-step-functions-and-amazon-api-gateway/>.

For more information about activities, see <https://docs.aws.amazon.com/step-functions/latest/dg/concepts-activities.html>.

Use case example (continued)

```
{
  "Comment": "Employee promotion process",
  "StartAt": "ManualApproval",
  "States": {
    "ManualApproval": {
      "Type": "Task",
      "Resource": "arn:aws:states:us-east-1:ACCOUNT_ID:activity:ManualStep",
      "TimeoutSeconds": 3600,
      "End": true
    }
  }
}
```



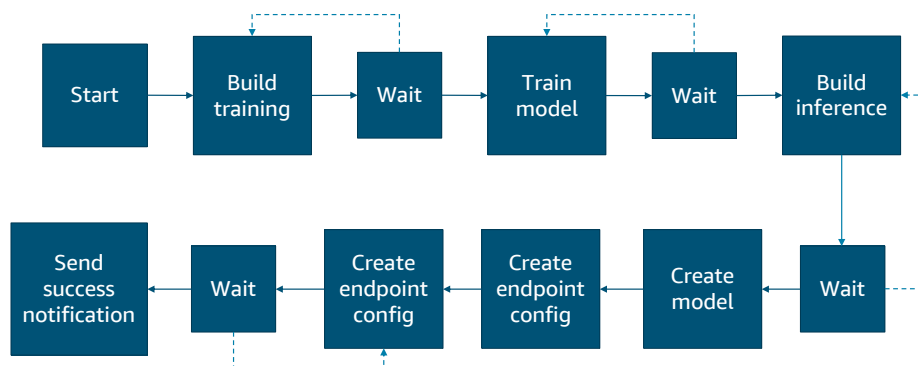
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

35

The JSON for the activity state machine for the manual approval process might look like this code. In this example, the state type is "Task" and the activity ARN is provided as a resource for the task.

For more information about how to integrate Step Functions with other AWS services, see <https://aws.amazon.com/step-functions/use-cases/>.

Step Functions use case example: Machine learning workflow



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

36

This diagram depicts the flow of a Step Functions state machine that is used for automated and continuous deployment of Amazon SageMaker models with Step Functions. The dashed arrows that are highlighted in pink are instances where the state machine must poll and wait for a task to be completed.

For more information, see the Automated and Continuous Deployment of Amazon SageMaker Models with AWS Step Functions blog post at <https://aws.amazon.com/blogs/machine-learning/automated-and-continuous-deployment-of-amazon-sagemaker-models-with-aws-step-functions/>.

Section 5 key takeaways



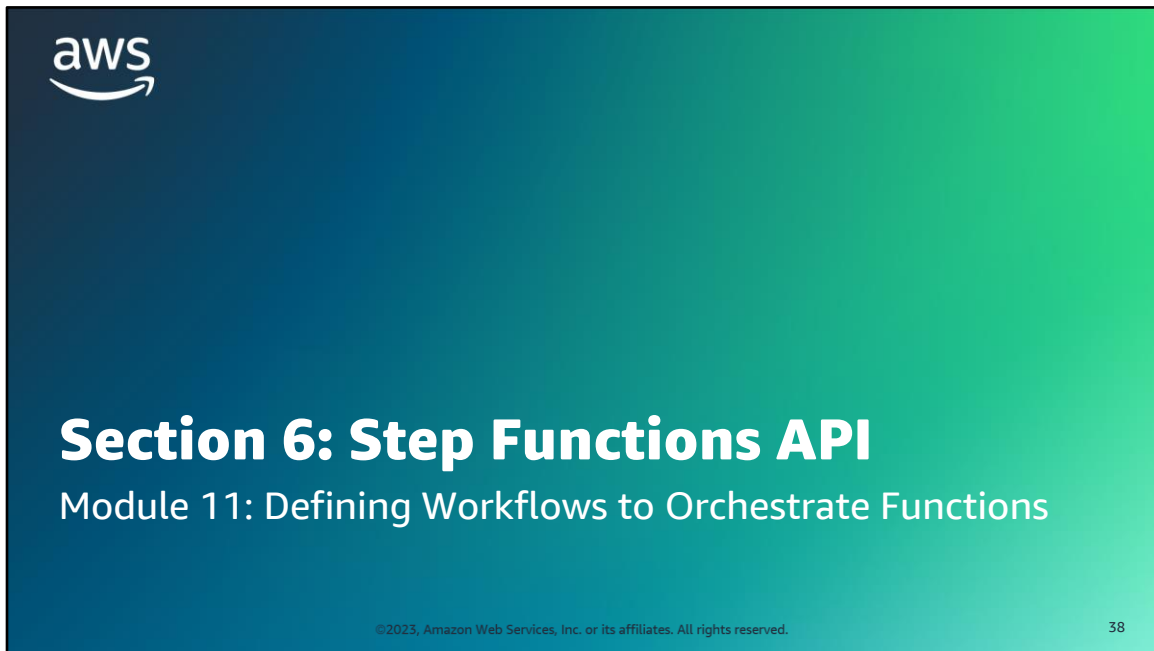
- **Activities** are a Step Functions feature that **associate code** with a **specific task** in a state machine.
- A **taskToken** is a token that represents a specific task that Step Functions generates when a task is assigned to a worker.
- An activity worker can run on any application that is able to make an **HTTP** connection.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

37

The following are the key takeaways from this section of the module:

- Activities are a Step Functions feature that associate code with a specific task in a state machine.
- A taskToken is a token that represents a specific task that Step Functions generates when a task is assigned to a worker.
- An activity worker can run on any application that is able to make an HTTP connection.



Section 6: Step Functions API.

Step Functions API

Create	Upload state machines that are defined in JSON. Register activity workers.
StartExecution	Return an Amazon Resource Name (ARN) that identifies the run.
StopExecution	Return the date that the state machine stopped.
List	List all state machines, runs, and activities.
Describe	Describe individual state machines, runs, and activities.
GetExecutionHistory	Get the history of a state machine run as a list of events.



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

39

You can access and use Step Functions by using the AWS Management Console, AWS SDKs, or API.

You can run a Step Functions state machine by doing the following:

- Call the Step Functions *StartExecution* API action.
- Associate your Step Functions APIs with API operations in API Gateway. When an HTTPS request is sent to an API operation, API Gateway invokes your Step Functions API actions.

For more information, see the following resources:

- *AWS Step Functions API Reference*: <https://docs.aws.amazon.com/step-functions/latest/apireference/Welcome.html>.
- *StartExecution*: https://docs.aws.amazon.com/step-functions/latest/apireference/API_StartExecution.html.
- *Creating a Step Functions API Using API Gateway*: <https://docs.aws.amazon.com/step-functions/latest/dg/tutorial-api-gateway.html>.
- *Periodically Start a State Machine Execution Using CloudWatch Events*: <https://docs.aws.amazon.com/step-functions/latest/dg/tutorial-cloudwatch-events-target.html>.

Step Functions quotas

- General quotas (for example, names in Step Functions)
- Quotas that are related to:
 - Accounts (for example, maximum number of registered state machines or API actions)
 - State machine runs (for example, maximum runtime)
 - Task runs (for example, maximum task runtime)
 - API action throttling
 - State throttling
- Quotas might differ between Standard and Express workflows



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

40

Step Functions places *quotas*, formerly referred to as limits, on the sizes of certain state machine parameters. These parameters might include the number of API actions you can make during a certain time period or how many state machines you can define. Although these quotas are designed to prevent a misconfigured state machine from consuming all of the resources of the system, they aren't hard quotas. Quotas might differ significantly based on whether your state machine uses Standard or Express workflows.

For more information about these quotas, see <https://docs.aws.amazon.com/step-functions/latest/dg/limits-overview.html>.

Section 6 key takeaways



- You can access Step Functions by using the **AWS Management Console**, **AWS SDKs**, or **API**.
- Step Functions places **quotas** on the size of certain state machine parameters.
- Use **API Gateway** to associate your Step Functions APIs with **methods** in an API Gateway API.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

41

The following are the key takeaways from this section of the module:

- You can access Step Functions by using the console, AWS SDKs, or API.
- Step Functions places quotas on the size of certain state machine parameters.
- Use API Gateway to associate your Step Functions APIs with methods in an API Gateway API.

Demonstration: Creating Simple Calculators by Using Step Functions

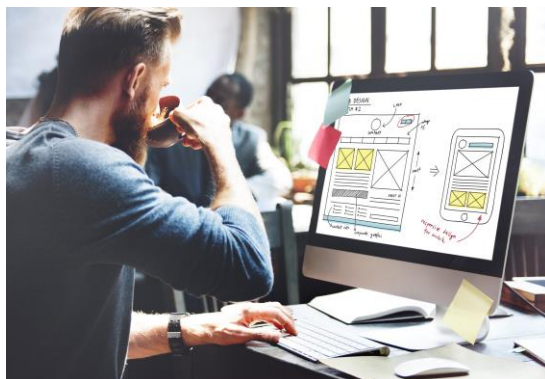


©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

42

There is a video demonstration available for this topic. You can find this video within the module 11 section of the course with the title: **Demo Creating Simple Calculators by Using Step Functions**. If you are unable to locate this video demonstration please reach out to your educator for assistance.

Lab 11.1: Orchestrating Serverless Functions with Step Functions



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

43

You will now complete Lab 11.1: Orchestrating Serverless Functions with Step Functions.

Lab: Scenario

In this lab, you will build the functionality to create and deliver the coffee inventory report using Step Functions.



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

44

In this lab, you will build functionality to create and deliver the coffee inventory report using Step Functions.

Lab: Tasks

1. Preparing the development environment
2. Configuring an SNS topic and subscribing to it
3. Creating a Step Functions state machine
4. Creating a Lambda function to generate a presigned URL
5. Configuring the REST API to invoke the state machine
6. Configuring a Lambda function to generate an HTML report
7. Adding the GenerateHTML function to the state machine
8. Creating a Lambda function to retrieve supplier records
9. Adding the generateReportData function to the state machine



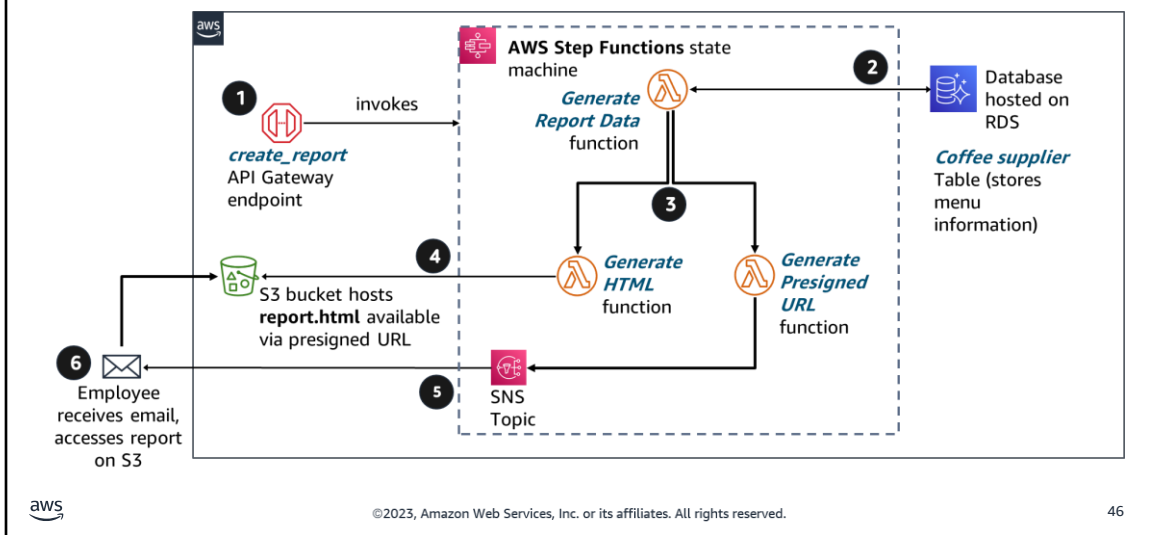
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

45

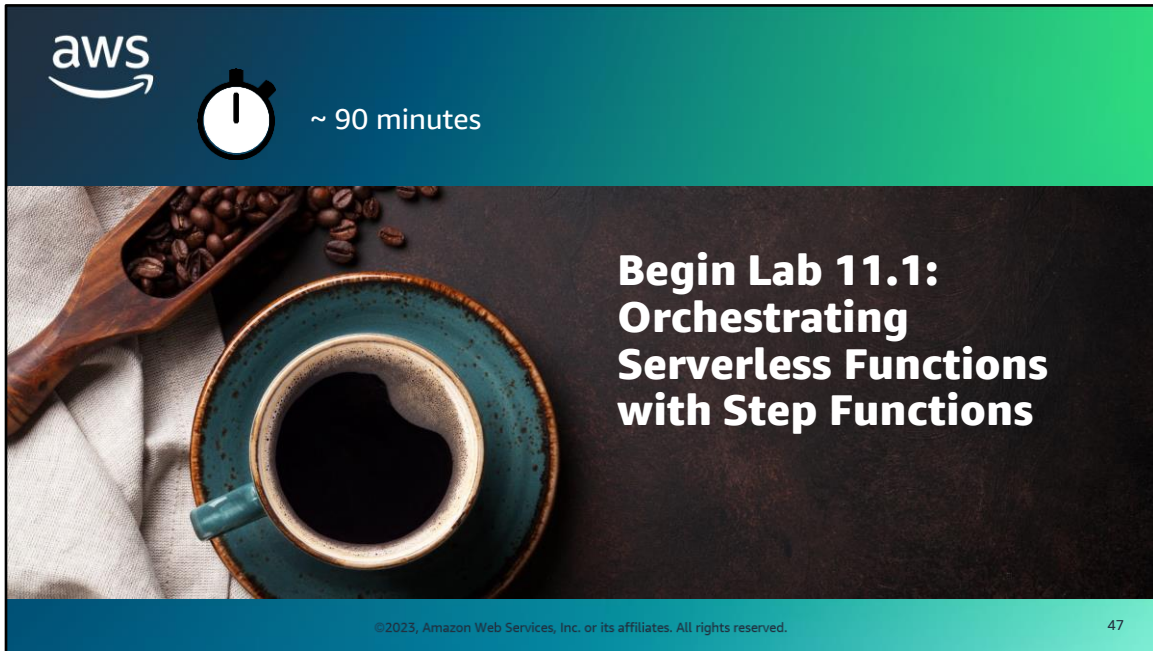
In this lab, you will complete the following tasks:

1. Preparing the development environment
2. Configuring an SNS topic and subscribing to it
3. Creating a Step Functions state machine
4. Creating a Lambda function to generate a presigned URL
5. Configuring the REST API to invoke the state machine
6. Configuring a Lambda function to generate an HTML report
7. Adding the GenerateHTML function to the state machine
8. Creating a Lambda function to retrieve supplier records
9. Adding the generateReportData function to the state machine

Lab: Final product



The diagram summarizes what you will have built after you complete the lab.



The image is a slide for an AWS lab. At the top, there is a dark blue header with the AWS logo on the left, a clock icon in the center, and the text '~ 90 minutes' on the right. Below the header is a large image of a coffee cup on a saucer with coffee beans scattered around it. To the right of the coffee image, the text 'Begin Lab 11.1: Orchestrating Serverless Functions with Step Functions' is displayed in white. At the bottom of the slide, there is a teal bar containing the copyright text '©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.' on the left and the number '47' on the right.

aws ~ 90 minutes

**Begin Lab 11.1:
Orchestrating
Serverless Functions
with Step Functions**

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved. 47

It is now time to start the lab.

Lab debrief: Key takeaways

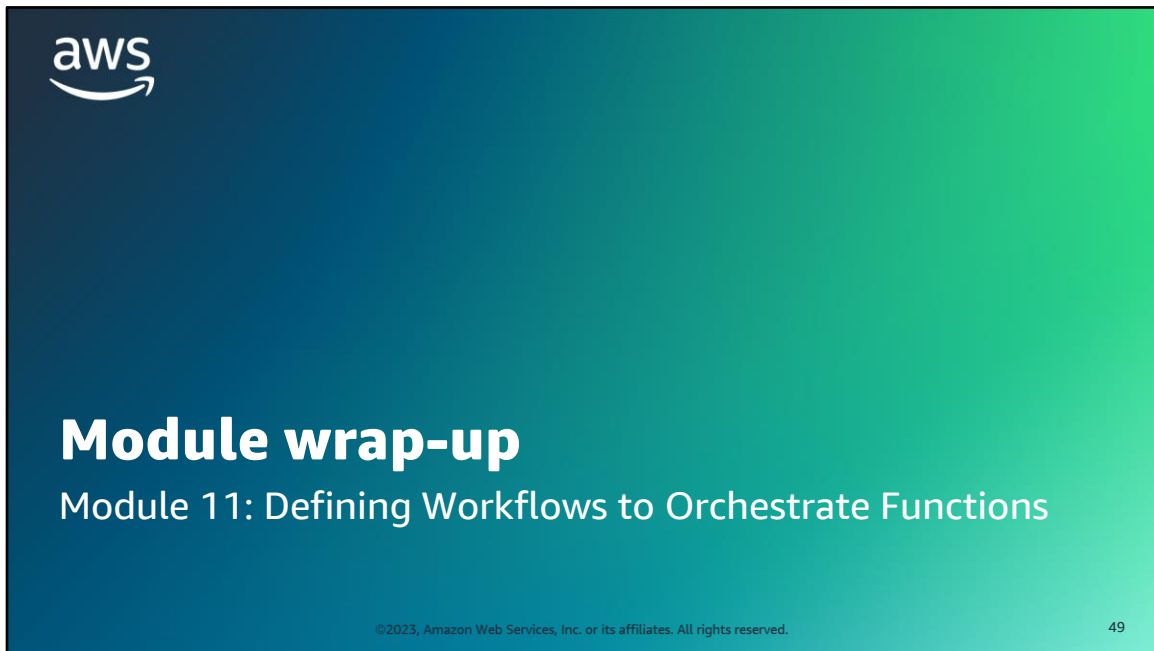


aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

48

Your educator might choose to lead a conversation about the key takeaways from this lab after you have completed it.



It's now time to review the module and wrap up with a knowledge check and discussion of a practice certification exam question.

Module summary

In summary, in this module, you learned how to:

- Recognize the dynamics of workflow coordination in distributed applications
- Describe Step Functions
- Identify state types
- Indicate common use cases for Step Functions
- Recall Step Functions API operations



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

50

In summary, in this module, you learned how to:

- Recognize the dynamics of workflow coordination in distributed applications
- Describe Step Functions
- Identify state types
- Indicate common use cases for Step Functions
- Recall Step Functions API operations

Complete the knowledge check



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

51

It is now time to complete the knowledge check for this module.

Sample exam question

When a customer submits a request for a credit limit increase, the request is to be processed automatically by comparing their request to their credit limit. If the request is more than their preapproved credit limit, Step Functions must send the request to a manager for approval.

Which type of state should be used to accomplish this process?

Identify the key words and phrases before continuing.

The following are the key words and phrases:

- Comparing
- Send the request to a manager



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

52

It is important to fully understand the scenario and question being asked before even reading the answer choices. Find the keywords in this scenario and question that will help you find the correct answer.

Sample exam question: Responses

When a customer submits a request for a credit limit increase, the request is to be processed automatically by **comparing** their request to their credit limit. If the request is more than their preapproved credit limit, Step Functions must **send the request to a manager** for approval.

Which type of state should be used to accomplish this process?

Choice	Response
A	Parallel
B	Choice
C	Task
D	Pass

 ©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved. 53

Now that we have bolded the keywords in this scenario, let us look at the answers.

Sample exam question: Answer

The correct answer is B.

Choice	Response
A	Parallel
B	Choice
C	Task
D	Pass



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

54

Look at the answer choices and rule them out based on the keywords that were previously highlighted.

The correct answer is B. Choice.

Using a Choice state, you can have Step Functions make decisions based on the Choice state's input.

Additional resources

- AWS Machine Learning Blog: [Automated and continuous deployment of Amazon SageMaker models with AWS Step Functions](#)
- AWS Compute Blog: [Implementing Serverless Manual Approval Steps in AWS Step Functions and Amazon API Gateway](#)
- AWS Technical Guide: [Create a Serverless Content Syndication Pipeline with AWS Step Functions](#)
- FAQ: [AWS Step Functions FAQs](#)

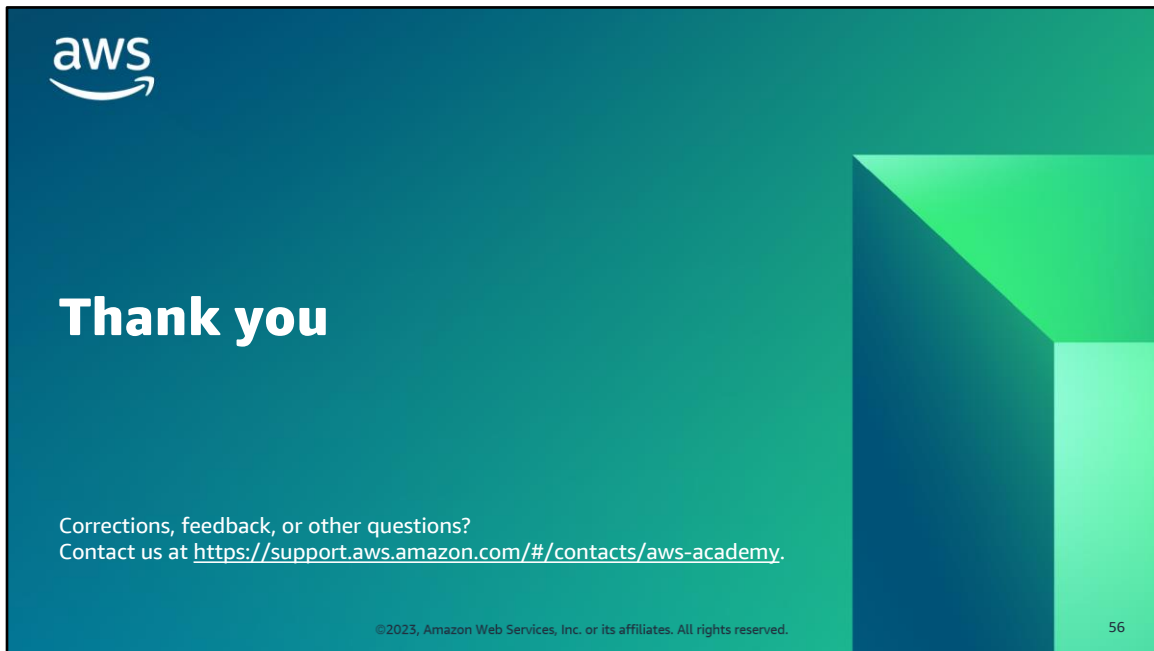


©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

55

To learn more about the topics covered in this module, you might find the following additional resources helpful:

- AWS Machine Learning Blog: <https://aws.amazon.com/blogs/machine-learning/automated-and-continuous-deployment-of-amazon-sagemaker-models-with-aws-step-functions/>.
- AWS Compute Blog: <https://aws.amazon.com/blogs/compute/implementing-serverless-manual-approval-steps-in-aws-step-functions-and-amazon-api-gateway/>.
- AWS Technical Guide: https://d1.awsstatic.com/whitepapers/create-a-serverless-content-syndication-pipeline-with-aws-step-functions.pdf?did=wp_card&trk=wp_card.
- FAQ: https://aws.amazon.com/premiumsupport/knowledge-center/?ref=docs_gateway#AWS Step Functions.



Thank you for completing this module.